

INFO145 - Diseño y Análisis de Algoritmos

Algoritmos En Línea

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

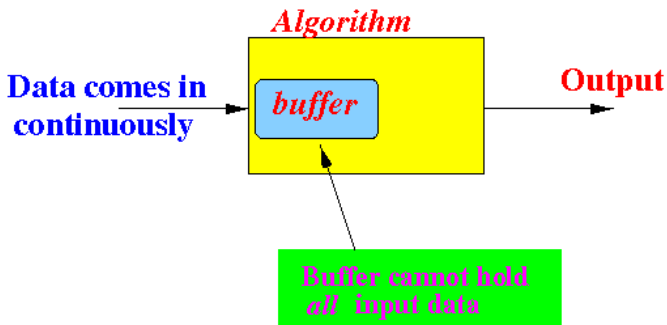
Algoritmos En Línea

- Son algoritmos que funcionan sin disponer de todos los datos de entrada al problema
- Los datos van llegando *on-line* (frecuentemente de apoco)
- Necesitan tomar desiciones "al vuelo" con la información que tengan disponible en el momento
- Es un enfoque opuesto al de los algoritmos *off-line*, los cuales requieren de toda la entrada para operar.

Algoritmos En Línea

- Son algoritmos que funcionan sin disponer de todos los datos de entrada al problema
- Los datos van llegando *on-line* (frecuentemente de apoco)
- Necesitan tomar desiciones "al vuelo" con la información que tengan disponible en el momento
- Es un enfoque opuesto al de los algoritmos *off-line*, los cuales requieren de toda la entrada para operar.
- Por ejemlo, la mayoría de los algoritmos de ordenamiento son *off-line*: quicksort, mergesort, heapsort y selectionsort, los que necesitan de todos los datos para ordenar. En cambio, insertionsort va tomando de a un elemento a la vez y no necesita de todos los elementos para operar.
⇒ insertionsort es un algoritmo *on-line*.

Algoritmos En Línea



Radio de Competitividad

Sea un algoritmo online $\mathcal{A}$, definimos su **radio de competitividad**, en el peor caso, como la razón:

$$\frac{C_{\mathcal{A}}}{C_{opt}} \geq 1$$

donde $C_{\mathcal{A}}$ es el costo del algoritmo $\mathcal{A}$ (costo de algún recurso) y C_{opt} es el costo óptimo para resolver el problema.

Ski Rental Problem

Arriendo diario de ski $\rightarrow \$a$

Comprar un par de skies $\rightarrow \$b$, suponga $b \ggg a$.

- Si sé cuántos días vendré a esquiar sabría qué es mejor: arrendar todos los días o comprarme un par.
- Costo óptimo $\rightarrow \min(a \cdot n, b)$ para los n días que vendré

Ski Rental Problem

Arriendo diario de ski $\rightarrow \$a$

Comprar un par de skies $\rightarrow \$b$, suponga $b \gg a$.

- Si sé cuántos días vendré a esquiar sabría qué es mejor: arrendar todos los días o comprarme un par.
- Costo óptimo $\rightarrow \min(a \cdot n, b)$ para los n días que vendré

Alg. 1: Decido comprar los ski's apenas llego.

Ski Rental Problem

Arriendo diario de ski $\rightarrow \$a$

Comprar un par de skis $\rightarrow \$b$, suponga $b \gg a$.

- Si sé cuántos días vendré a esquiar sabría qué es mejor: arrendar todos los días o comprarme un par.
- Costo óptimo $\rightarrow \min(a \cdot n, b)$ para los n días que vendré

Alg. 1: Decido comprar los ski's apenas llego.

Pero un peor adversario decide que venga solo un día $\Rightarrow C\mathcal{A} = b \gg a$

$\therefore$ **Alg. 1** es $\frac{b}{a}$ competitivo.

Por ejemplo, si $b = \$300$ y $a = \$15 \Rightarrow$ el ratio de competitividad es $\frac{300}{15} = 20$; es decir, es 20 veces peor que la solución óptima.

Ski Rental Problem

Alg. 2: Decido arrendar siempre

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C\mathcal{A} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C\mathcal{A} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C\mathcal{A}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} \end{cases}$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \left\{ a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \right. \end{cases}$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} \end{cases} \end{cases}$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \end{cases} \end{cases} \quad (1)$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C\mathcal{A} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C\mathcal{A}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} \end{cases} \quad (1)$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} \end{cases} \end{cases} \quad (1)$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \end{cases} (1) \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} < \frac{a \cdot k + b}{a \cdot k} = 1 + \frac{b}{a \cdot k} \end{cases} (2) \end{cases}$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C\mathcal{A} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C\mathcal{A}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \quad (1) \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} < \frac{a \cdot k + b}{a \cdot k} = 1 + \frac{b}{a \cdot k} \quad (2) \\ a \cdot n > b : \frac{a \cdot k + b}{b} = 1 + \frac{a \cdot k}{b} \quad (3) \end{cases} \end{cases}$$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C\mathcal{A} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C\mathcal{A}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \quad (1) \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} < \frac{a \cdot k + b}{a \cdot k} = 1 + \frac{b}{a \cdot k} \quad (2) \\ a \cdot n > b : \frac{a \cdot k + b}{b} = 1 + \frac{a \cdot k}{b} \quad (3) \end{cases} \end{cases}$$

¿Qué valor de k minimiza las tres razones?

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{opt}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \quad (1) \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} < \frac{a \cdot k + b}{a \cdot k} = 1 + \frac{b}{a \cdot k} \quad (2) \\ a \cdot n > b : \frac{a \cdot k + b}{b} = 1 + \frac{a \cdot k}{b} \quad (3) \end{cases} \end{cases}$$

¿Qué valor de k minimiza las tres razones? $\Rightarrow k = \frac{b}{a}$

Ski Rental Problem

Alg. 2: Decido arrendar siempre, pero si $n \ggg$ este algoritmo es infinitamente peor que el óptimo

Alg. 3 Arriendo por k días y luego compro.

$$C_{\mathcal{A}} = \begin{cases} a \cdot n & \text{si } n \leq k \\ a \cdot k + b & \text{si } n > k \end{cases}$$

¿Qué tan competitivo es este algoritmo?

$$\frac{C_{\mathcal{A}}}{C_{\text{opt}}} = \begin{cases} n \leq k \Rightarrow \frac{a \cdot n}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot n}{a \cdot n} = 1 \\ a \cdot n > b : \frac{a \cdot n}{b} < \frac{a \cdot k}{b} \quad (1) \end{cases} \\ n > k \Rightarrow \frac{a \cdot k + b}{\min(a \cdot n, b)} & \begin{cases} a \cdot n \leq b : \frac{a \cdot k + b}{a \cdot n} < \frac{a \cdot k + b}{a \cdot k} = 1 + \frac{b}{a \cdot k} \quad (2) \\ a \cdot n > b : \frac{a \cdot k + b}{b} = 1 + \frac{a \cdot k}{b} \quad (3) \end{cases} \end{cases}$$

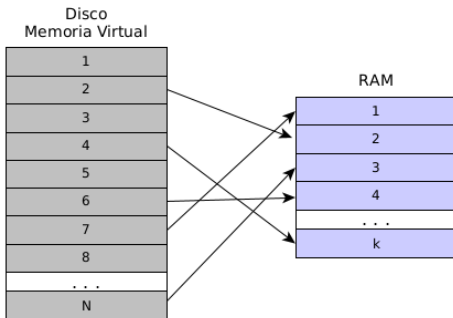
¿Qué valor de k minimiza las tres razones? $\Rightarrow k = \frac{b}{a}$

$\therefore$ En el peor caso nuestro **Alg. 3** es 2-competitivo, siendo la estrategia arrendar $\frac{b}{a}$ días y luego comprar los ski's

Paginamiento

Se tiene un disco con N páginas pero la RAM tiene una capacidad para $k < N$ y hay una secuencia de requerimientos de páginas:

$$R = \{r_1, r_2, r_3, \dots, r_n\}.$$



Si r_i no está en la RAM cuando es solicitado: $\Rightarrow$ Ocurre un fallo ya que se debe cargar desde el disco a la memoria

$\Rightarrow$ Tenemos que decidir que página sacamos de la RAM para cargar r_i sin saber cuál es la página que será requerida en el siguiente paso.

Paginamiento

Objetivo: Diseñar una estrategia que minimize el número de fallos.

Estrategia óptima: Elegir la página para la cual falta más tiempo para que vuelva a ser solicitada.

Estrategias online (*Page Replacement Algorithms*):

- **Alg. LRU**¹ Sacar la página menos usada recientemente
- **Alg. MRU**² Sacar la página más usada recientemente
- **Alg. FIFO** Sacar la página que ha estado más tiempo en la RAM

¹Least Recently Used

²Most Recently Used

Paginamiento

Objetivo: Diseñar una estrategia que minimize el número de fallos.

Estrategia óptima: Elegir la página para la cual falta más tiempo para que vuelva a ser solicitada.

Estrategias online (*Page Replacement Algorithms*):

- **Alg. LRU**¹ Sacar la página menos usada recientemente
- **Alg. MRU**² Sacar la página más usada recientemente
- **Alg. FIFO** Sacar la página que ha estado más tiempo en la RAM

Observación: en todo algoritmo online para paginamiento siempre existe una secuencia de n requerimientos que produce n fallos de página. Esto se demuestra con la estrategia del peor adversario, el cual pondrá siempre un requerimiento de página que no esta en RAM.

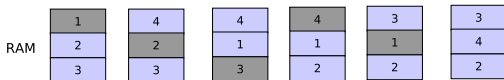
¹Least Recently Used

²Most Recently Used

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

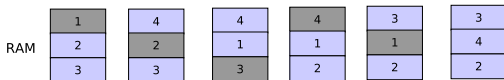
Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

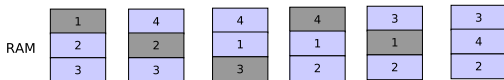
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

| 1 2 3 | r: 4

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

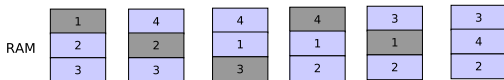
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

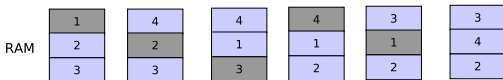
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

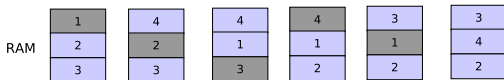
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

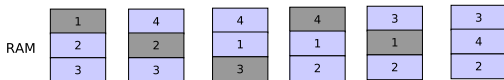
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

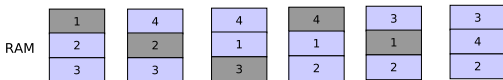
$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

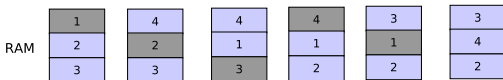
$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

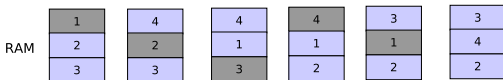
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3| r: 4 \Rightarrow |2\ 3\ 4| r: 1 \Rightarrow |3\ 4\ 1| r: 2 \Rightarrow |4\ 1\ 2| r: 3 \Rightarrow$
 $|1\ 2\ 3| r: 4 \Rightarrow |2\ 3\ 4| r: 1 \Rightarrow |3\ 4\ 1| r: 2 \Rightarrow |4\ 1\ 2| r: 3$

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 | r: 1 $\Rightarrow$ | 3 4 1 | r: 2 $\Rightarrow$ | 4 1 2 | r: 3 $\Rightarrow$

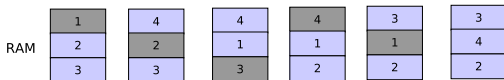
| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 | r: 1 $\Rightarrow$ | 3 4 1 | r: 2 $\Rightarrow$ | 4 1 2 | r: 3 $\Rightarrow$

| 1 2 3 | r: 4

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

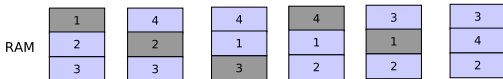
$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 | r: 1 $\Rightarrow$ | 3 4 1 | r: 2 $\Rightarrow$ | 4 1 2 | r: 3 $\Rightarrow$
| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 | r: 1 $\Rightarrow$ | 3 4 1 | r: 2 $\Rightarrow$ | 4 1 2 | r: 3 $\Rightarrow$
| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 | r: 1 $\Rightarrow$ | 3 4 1 | r: 2 $\Rightarrow$ | 4 1 2 | r: 3 $\Rightarrow$
| 1 2 3 | r: 4 $\Rightarrow$ | 2 3 4 |

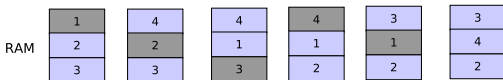
Fallos de LRU: $m_{LRU} = n - k$ en el peor caso

Fallos del óptimo $m_{OPT} = n/k - 1$ en el peor caso, ya que tendrá a lo más 1 fallo cada k requerimientos.

Paginamiento - LRU

Alg. LRU. Sacar la página menos usada recientemente

Ejemplo: $R = \{1, 2, 3, 4, 3, 4, 1, 2, 3, 4\}$, con $k = 3$



Worst Case LRU:

$R = \{1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4\}$, con $k = 3$

$|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$ r: 1 $\Rightarrow$ $|3\ 4\ 1|$ r: 2 $\Rightarrow$ $|4\ 1\ 2|$ r: 3 $\Rightarrow$
 $|1\ 2\ 3|$ r: 4 $\Rightarrow$ $|2\ 3\ 4|$

Fallos de LRU: $m_{LRU} = n - k$ en el peor caso

Fallos del óptimo $m_{OPT} = n/k - 1$ en el peor caso, ya que tendrá a lo más 1 fallo cada k requerimientos.

Competitividad $\frac{n-k}{n/k-1} = \frac{n-k}{(n-k)/k} = k$

$\therefore$ LRU es k competitivo

Load Balancing

- Se tienen m máquinas idénticas que pueden operar en paralelo.
- Se dan n tareas que deben ser asignadas entre las máquinas
- Cada tarea t_k añade una carga de trabajo $p_k > 0$ a la máquina a la cual es asignada

Objetivo: distribuir las n tareas de tal forma que minimice la carga de trabajo de la máquina más sobrecargada.

Load Balancing

- Se tienen m máquinas idénticas que pueden operar en paralelo.
- Se dan n tareas que deben ser asignadas entre las máquinas
- Cada tarea t_k añade una carga de trabajo $p_k > 0$ a la máquina a la cual es asignada

Objetivo: distribuir las n tareas de tal forma que minimice la carga de trabajo de la máquina más sobrecargada.

Notación: Llamaremos a $L_i(j)$ a la carga acumulada de la máquina i una vez asignadas las primeras j tareas.

Load Balancing

- Se tienen m máquinas idénticas que pueden operar en paralelo.
- Se dan n tareas que deben ser asignadas entre las máquinas
- Cada tarea t_k añade una carga de trabajo $p_k > 0$ a la máquina a la cual es asignada

Objetivo: distribuir las n tareas de tal forma que minimice la carga de trabajo de la máquina más sobrecargada.

Notación: Llamaremos a $L_i(j)$ a la carga acumulada de la máquina i una vez asignadas las primeras j tareas.

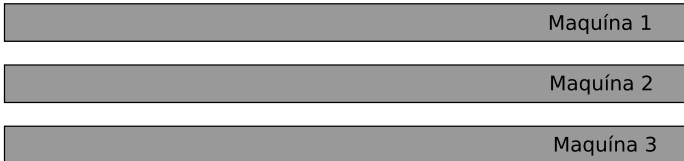
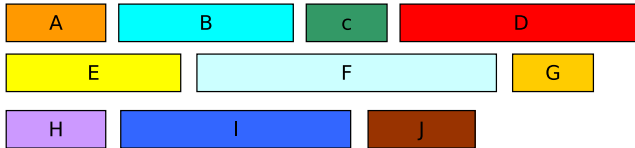
$$\Rightarrow L_i(j) = \begin{cases} L_i(j-1) + p_j & \text{si } t_j \text{ se asigna a } m_i \\ L_i(j-1) & \text{e.o.c} \end{cases}$$

Tarea: escriba el pseudocódigo del algoritmo exhaustivo que resuelve este problema y determine su tiempo asintótico de ejecución.

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

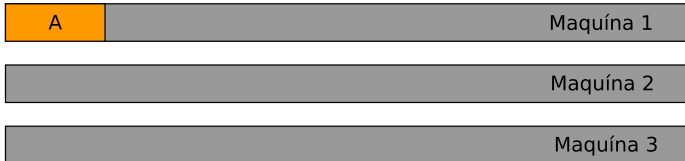
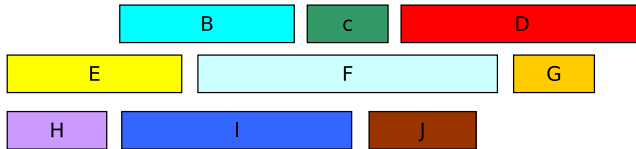
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

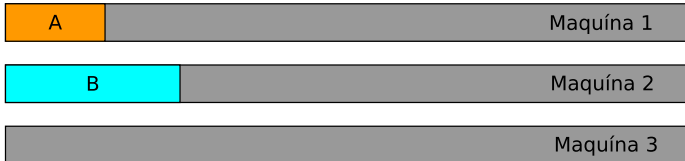
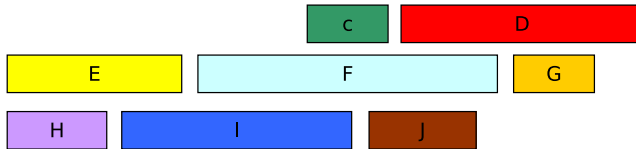
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

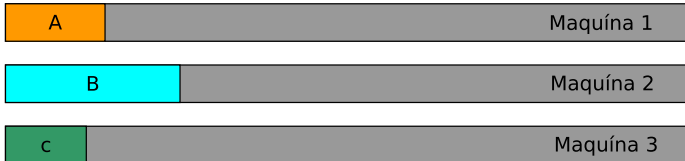
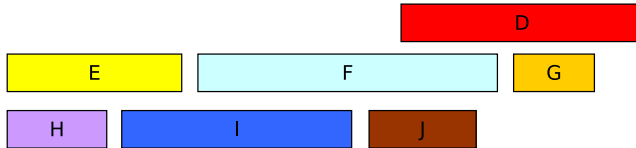
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

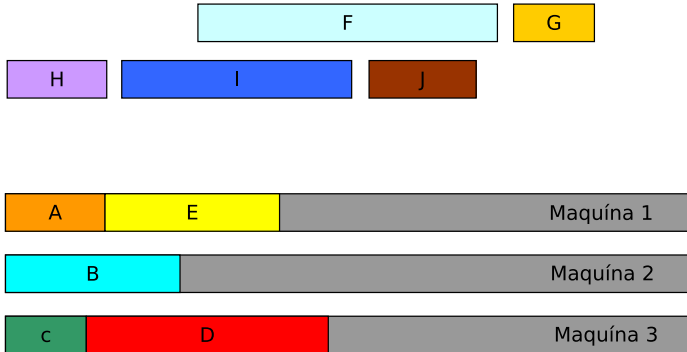
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

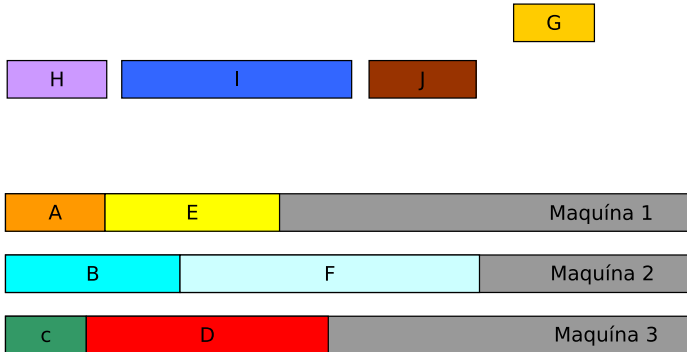
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

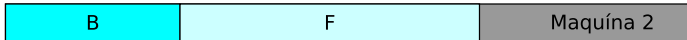
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

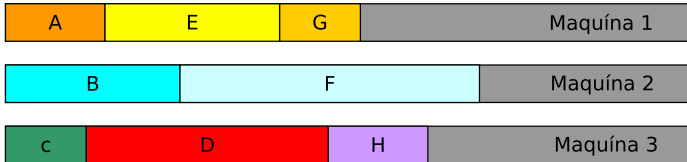
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Tareas:

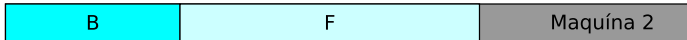


Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Tareas:

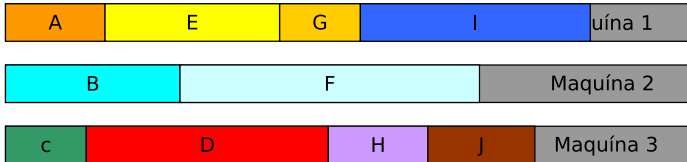
J



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

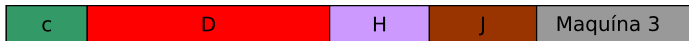
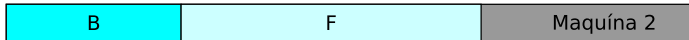
Tareas:



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Optimo



Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

Dem. Sea L_j la carga acumulada de la máquina j

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

Dem. Sea L_j la carga acumulada de la máquina j

Por el absurdo: suponga que, para algún k , al asignar t_k a la máquina i (la menos cargada), ocurre que $L_i > 2L^*$, donde L^* es el óptimo del algoritmo offline.

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

Dem. Sea L_j la carga acumulada de la máquina j

Por el absurdo: suponga que, para algún k , al asignar t_k a la máquina i (la menos cargada), ocurre que $L_i > 2L^*$, donde L^* es el óptimo del algoritmo offline.

$\Rightarrow$ Cómo $p_k \leq L^*$, $\forall k \leq n$, **necesariamente todas las m máquinas**, justo antes de la tarea k , tienen una carga $> L^*$ y la máquina i tiene la menor de las cargas³.

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto $\equiv$

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

Dem. Sea L_j la carga acumulada de la máquina j

Por el absurdo: suponga que, para algún k , al asignar t_k a la máquina i (la menos cargada), ocurre que $L_i > 2L^*$, donde L^* es el óptimo del algoritmo offline.

$\Rightarrow$ Cómo $p_k \leq L^*$, $\forall k \leq n$, **necesariamente todas las m máquinas**, justo antes de la tarea k , tienen una carga $> L^*$ y la máquina i tiene la menor de las cargas³.

$$\Rightarrow \sum_{j=1}^m L_j > m \cdot L^*, \text{ es decir la carga promedio } \frac{\sum_{j=1}^m L_j}{m} > L^*$$

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto $\equiv$

Load Balancing - List Scheduling

Alg. Greedy List Scheduling: Asignar la tarea a la máquina con menor carga acumulada.

Teorema [Graham'66]. Greedy List Scheduling es 2-Competitivo.

Dem. Sea L_j la carga acumulada de la máquina j

Por el absurdo: suponga que, para algún k , al asignar t_k a la máquina i (la menos cargada), ocurre que $L_i > 2L^*$, donde L^* es el óptimo del algoritmo offline.

$\Rightarrow$ Cómo $p_k \leq L^*$, $\forall k \leq n$, **necesariamente todas las m máquinas**, justo antes de la tarea k , tienen una carga $> L^*$ y la máquina i tiene la menor de las cargas³.

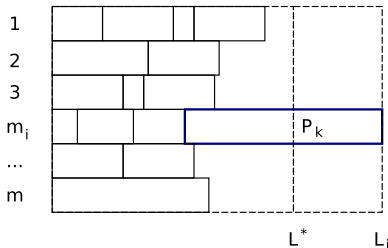
$$\Rightarrow \sum_{j=1}^m L_j > m \cdot L^*, \text{ es decir la carga promedio } \frac{\sum_{j=1}^m L_j}{m} > L^* \rightarrow \leftarrow$$

Contradicción, ya que el promedio de carga no puede superar a la máquina más sobrecargada; de se así, no se puede alcanzar el óptimo L^*

³si no, entonces el alg. online no puede tener el costo $> 2L^*$ del supuesto $\equiv$

Load Balancing - List Scheduling

- ¿Es posible encontrar una cota más ajustada?

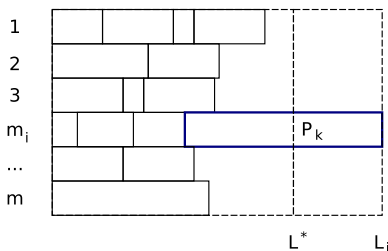


En la imagen, suponga que la carga mayor es justo la última $p_k = \max_{1 \leq i \leq n} p_i$.

$$\Rightarrow m \cdot L_i \leq \sum_{i=1}^n p_i + (m-1) \max_{1 \leq i \leq n} p_i$$

Load Balancing - List Scheduling

- ¿Es posible encontrar una cota más ajustada?



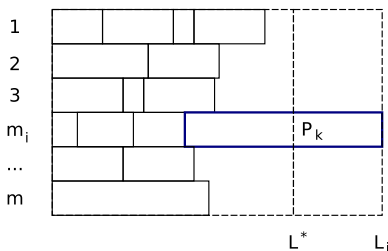
En la imagen, suponga que la carga mayor es justo la última $p_k = \max_{1 \leq i \leq n} p_i$.

$$\Rightarrow m \cdot L_i \leq \sum_{i=1}^n p_i + (m-1) \max_{1 \leq i \leq n} p_i$$

$$\Rightarrow L_i \leq \frac{1}{m} \sum_{i=1}^n p_i + \left(1 - \frac{1}{m}\right) \max_{1 \leq i \leq n} p_i$$

Load Balancing - List Scheduling

- ¿Es posible encontrar una cota más ajustada?



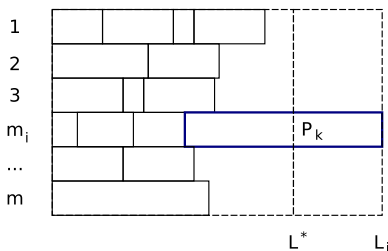
En la imagen, suponga que la carga mayor es justo la última $p_k = \max_{1 \leq i \leq n} p_i$.

$$\Rightarrow m \cdot L_i \leq \sum_{i=1}^n p_i + (m-1) \max_{1 \leq i \leq n} p_i$$

$$\Rightarrow L_i \leq \frac{1}{m} \sum_{i=1}^n p_i + \left(1 - \frac{1}{m}\right) \max_{1 \leq i \leq n} p_i \leq L^* + \left(1 - \frac{1}{m}\right) L^*$$

Load Balancing - List Scheduling

- ¿Es posible encontrar una cota más ajustada?



En la imagen, suponga que la carga mayor es justo la última $p_k = \max_{1 \leq i \leq n} p_i$.

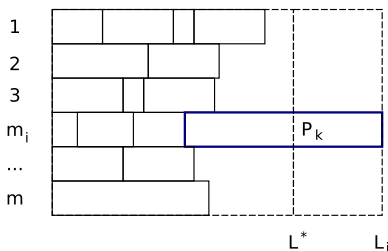
$$\Rightarrow m \cdot L_i \leq \sum_{i=1}^n p_i + (m-1) \max_{1 \leq i \leq n} p_i$$

$$\Rightarrow L_i \leq \frac{1}{m} \sum_{i=1}^n p_i + \left(1 - \frac{1}{m}\right) \max_{1 \leq i \leq n} p_i \leq L^* + \left(1 - \frac{1}{m}\right) L^*$$

$$\Rightarrow \frac{L_i}{L^*} \leq 2 - \frac{1}{m}$$

Load Balancing - List Scheduling

- ¿Es posible encontrar una cota más ajustada?



En la imagen, suponga que la carga mayor es justo la última $p_k = \max_{1 \leq i \leq n} p_i$.

$$\Rightarrow m \cdot L_i \leq \sum_{i=1}^n p_i + (m-1) \max_{1 \leq i \leq n} p_i$$

$$\Rightarrow L_i \leq \frac{1}{m} \sum_{i=1}^n p_i + \left(1 - \frac{1}{m}\right) \max_{1 \leq i \leq n} p_i \leq L^* + \left(1 - \frac{1}{m}\right) L^*$$

$$\Rightarrow \frac{L_i}{L^*} \leq 2 - \frac{1}{m}$$

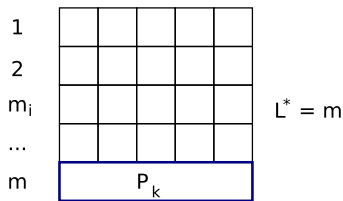
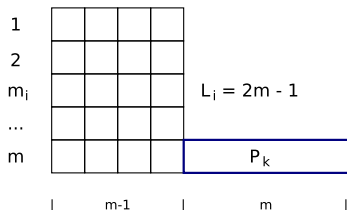
$\therefore$ En realidad, Greedy List Scheduling es $(2 - \frac{1}{m})$ competitivo.

Load Balancing - List Scheduling

- **Teorema.** Greedy List Scheduling no puede ser mejor que $(2 - \frac{1}{m})$ competitivo.

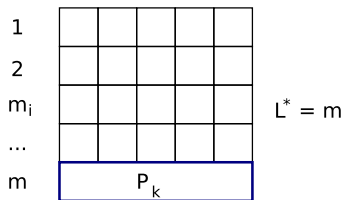
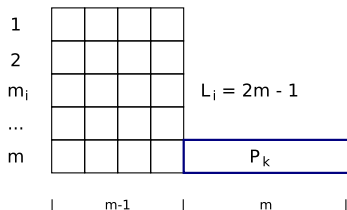
Load Balancing - List Scheduling

- **Teorema.** Greedy List Scheduling no puede ser mejor que $(2 - \frac{1}{m})$ competitivo.



Load Balancing - List Scheduling

- **Teorema.** Greedy List Scheduling no puede ser mejor que $(2 - \frac{1}{m})$ competitivo.



Tarea: formalizar la demostración en base a este contraejemplo.

Move to Front

Sea $S[1..n]$ una secuencia de n símbolos pertenecientes a un alfabeto $\Sigma[1..m]$. **Move to Front (MTF)** de S entrega una secuencia $T[1..n]$, $T[i] \in [1..m]$ (que es una transformación de S) que: *i.* requiere de menos espacio que S y/o *ii.* es más fácil de comprimir

El algoritmo **MTF** funciona de la siguiente manera:

Move to Front

Sea $S[1..n]$ una secuencia de n símbolos pertenecientes a un alfabeto $\Sigma[1..m]$. **Move to Front (MTF)** de S entrega una secuencia $T[1..n]$, $T[i] \in [1..m]$ (que es una transformación de S) que: *i.* requiere de menos espacio que S y/o *ii.* es más fácil de comprimir

El algoritmo **MTF** funciona de la siguiente manera:

- En una lista $L[1..m]$ almacenamos los m ID's de los símbolos, $L[i] \in \mathbb{Z}^+$
- Para cada símbolo en S , MTF codifica $S[i]$ con la posición de $S[i]$ en L y mueve el ID de $S[i]$ al frente de L .

Ejemplo. $S = \{bcaacaebae\}$, con $\Sigma[1..m] = \{a, b, c, d, e, \}$, $m = 5$:

Move to Front

Sea $S[1..n]$ una secuencia de n símbolos pertenecientes a un alfabeto $\Sigma[1..m]$.

Move to Front (MTF) de S entrega una secuencia $T[1..n]$, $T[i] \in [1..m]$ (que es una transformación de S) que: *i.* requiere de menos espacio que S y/o *ii.* es más fácil de comprimir

El algoritmo **MTF** funciona de la siguiente manera:

- En una lista $L[1..m]$ almacenamos los m ID's de los símbolos, $L[i] \in \mathbb{Z}^+$
- Para cada símbolo en S , MTF codifica $S[i]$ con la posición de $S[i]$ en L y mueve el ID de $S[i]$ al frente de L .

Ejemplo. $S = \{bcaacaebae\}$, con $\Sigma[1..m] = \{a, b, c, d, e, \}$, $m = 5$:

S:	b	c	a	a	c	a	e	b	a	e	
L: (1)	a	b	c	a	a	c	a	e	b	a	e
(2)	b	a	b	c	c	a	c	a	e	b	a
(3)	c	c	a	b	b	b	b	c	a	e	b
(4)	d	d	d	d	d	d	d	b	c	c	c
(5)	e	e	e	e	e	e	e	d	d	d	d
T:	2	3	3	1	2	2	5	4	3	3	

T se puede decodificar desde $T[n]$ hasta $T[a]$ usando el L final y aplicando el mismo algoritmo.

Move To Front

Input: La secuencia $S[1..n]$, $1 \leq S[i] \leq m$ (que llega de apoco) y la lista inicial $L[1..m]$

Output: La secuencia codificada $T[1..n]$ y la lista final $L[1..m]$

MoveToFront($S_{1..n}, P_{1..m}$) {

 Sea $T[1..n]$ un arreglo y $L[1..m]$ la lista inicial con los m ID's crecientes

 for $i = 1$ to n do {

$u = S[i]$

$T[i] = \text{position}(u, L)$ //corresponde a la posición de u en L

$k = u$

 while ($k > 1$) do {

$L[k] = L[k - 1]$

$k = k - 1$

 }

$L[1] = u$

 }

 return (T, L)

}

Tarea: escriba el pseudocódigo para la decodificación de T .

Move to Front - Análisis

- Todo algoritmo online $\mathcal{A}$ para Move To Front tiene un costo $CA = m \cdot n$, pues un peor adversario podría poner una secuencia que nos obligue a recorrer toda la lista en cada paso.
- El algoritmo óptimo off-line elige la mejor lista de entrada (hay $m!$ permutaciones); pero aún así no puede haber una estrategia óptima que baje el costo $m + (n - m) \cdot m$, ya que la secuencia a codificar podría poner la lista L en un peor caso.

Ejemplo:

$S[1..n] = 1, 2, 3, \dots, m, 1, 2, 3, \dots, m, 1, 2, 3, \dots, m, 1, 2, 3, \dots, m, 1, 2, 3, \dots$

Entonces solo para los primeros m símbolos un algoritmo óptimo puede encontrar una buena lista L , pero a partir del $(m + 1)$ -ésimo símbolo ya no podrá hacer nada.